

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Thời gian thực hiện: 01/10 – 10/10/2026

Sinh viên thực hiện: Nguyễn Hoàng Phúc – MSSV 24521388

Nội dung báo cáo:

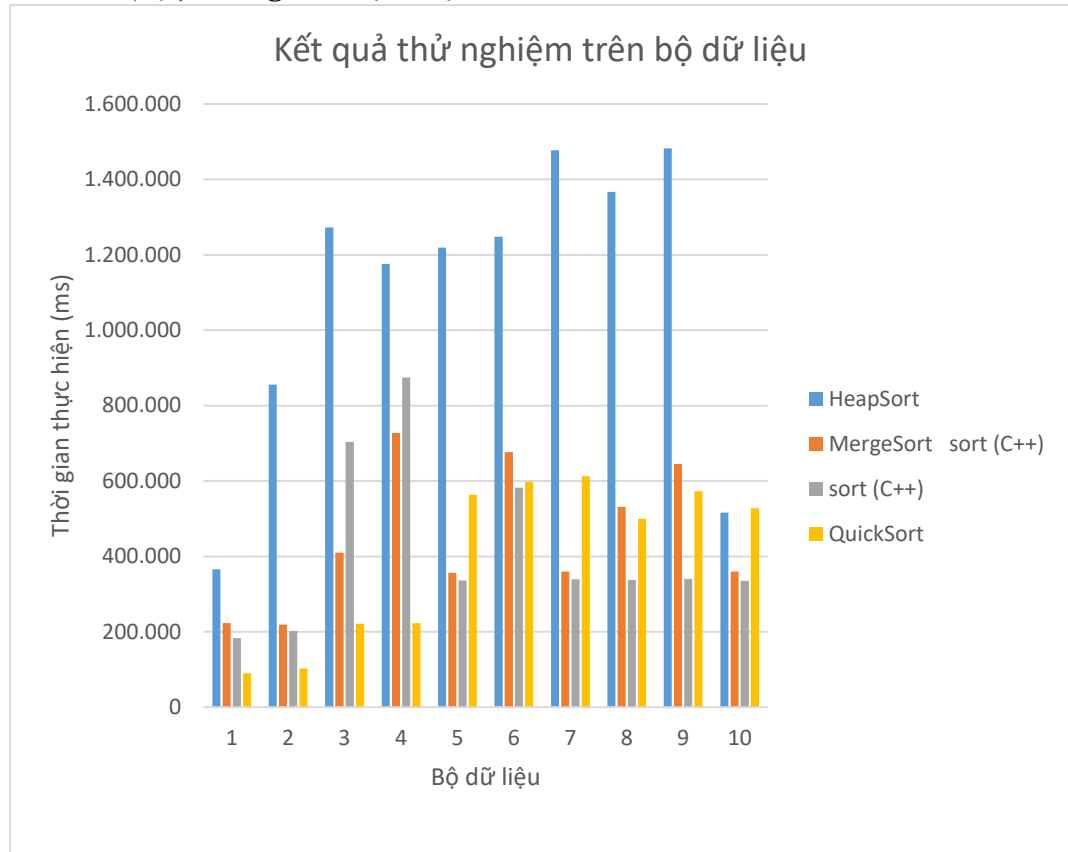
I. Kết quả thử nghiệm

1. Bảng thời gian thực hiện¹

	Thời gian thực hiện (ms)			
	Quicksort	Heapsort	Mergesort	sort (C++)
1	89.338	365.522	222.921	183.247
2	102.790	856.102	219.200	201.872
3	221.058	1272.735	409.730	703.289
4	222.994	1175.773	727.350	874.025
5	563.734	1218.778	356.157	336.101
6	597.834	1248.371	676.589	582.695
7	613.139	1477.079	359.534	339.178
8	500.008	1367.148	531.497	337.371
9	573.011	1482.755	644.957	340.336
10	527.854	515.782	359.552	335.184
Trung bình	401.176	1098.005	450.749	423.330

¹ Số liệu chỉ mang tính minh họa

2. Biểu đồ (cột) thời gian thực hiện



II. Kết luận:

Từ kết quả thực nghiệm trên 10 bộ dữ liệu (mỗi bộ 1.000.000 số thực), có thể rút ra các nhận xét sau:

1. Về hiệu năng tổng thể:

QuickSort đạt thời gian trung bình nhanh nhất (401.176 ms), theo sau là `std::sort` (423.330 ms) và MergeSort (450.749 ms). HeapSort chậm nhất với 1098.005 ms — gấp 2.74 lần so với QuickSort. Điều này phù hợp với lý thuyết: cả QuickSort, `std::sort` và MergeSort đều có độ phức tạp trung bình $O(n \log n)$ với hằng số nhỏ, trong đó QuickSort tận dụng tốt bộ nhớ đệm (cache-friendly). HeapSort tuy cũng là $O(n \log n)$ nhưng có mẫu truy cập bộ nhớ phân tán (nhảy xa trong mảng) nên kém hiệu quả trên thực tế.

2. Về ảnh hưởng của dữ liệu đầu vào:

- Dữ liệu 1 (tăng dần) và 2 (giảm dần): QuickSort, MergeSort và `std::sort` đều rất nhanh (~90–220 ms) vì chọn pivot giữa nên cây đệ quy vẫn cân bằng. HeapSort chậm hơn nhiều (~365–856 ms) do phải xây heap trên dữ liệu đã có thứ tự.
- Dữ liệu 3 → 10 (ngẫu nhiên): QuickSort vẫn dẫn đầu (~220–613 ms). MergeSort và `std::sort` dao động mạnh (340–880 ms), phụ thuộc đặc tính phân bố cục bộ của từng dãy ngẫu nhiên. HeapSort luôn nằm trong khoảng ~515–1483 ms.

3. Về tính ổn định:

- HeapSort có thời gian ổn định nhất về mặt lý thuyết (luôn $O(n \log n)$), nhưng lại chậm nhất về giá trị tuyệt đối. Ngoại lệ là dữ liệu 10 (515 ms) — có thể do dữ liệu có cấu trúc đặc biệt hoặc file có kích thước khác thường.
- MergeSort và `std::sort` có phương sai lớn (thời gian dao động đáng kể giữa các dãy ngẫu nhiên), thể hiện qua độ lệch giữa giá trị nhỏ nhất và lớn nhất.
- QuickSort giữ mức thời gian tương đối ổn định trên dữ liệu ngẫu nhiên (~500–613 ms), chỉ nhanh đột biến trên dữ liệu đã sắp xếp.

4. Về lựa chọn thuật toán:

Trong 4 thuật toán khảo sát, QuickSort (với pivot giữa) là lựa chọn tốt nhất về mặt hiệu năng tổng thể cho tập dữ liệu số thực. Tuy nhiên, trong thực tế lập trình, nên ưu tiên sử dụng `std::sort` của C++ vì:

- Hiệu năng tương đương QuickSort tự viết.
- Đã được tối ưu hóa (introsort = QuickSort + HeapSort + Insertion Sort).
- An toàn trước worst case $O(n^2)$ mà QuickSort thuần có thể gặp.
- Ngắn gọn, ít lỗi, dễ bảo trì.

5. Hạn chế của thực nghiệm:

- Thời gian đo phụ thuộc cấu hình máy (CPU, RAM, compiler, chế độ build).
- Chưa khảo sát ảnh hưởng của bộ nhớ đệm (cache) và bộ nhớ ảo (swap).
- MergeSort tự cài đặt tốn thêm bộ nhớ $O(n)$ cho mảng tạm, có thể ảnh hưởng khi RAM hạn chế.

III. Thông tin chi tiết – link github, trong repo gibub cần có

1. Báo cáo
2. Mã nguồn
3. Dữ liệu thử nghiệm